

CS & IT ENGINEERING

DATA STRUCTURE

Binary Search Tree



JOIN TELEGRAM TO ENROLL

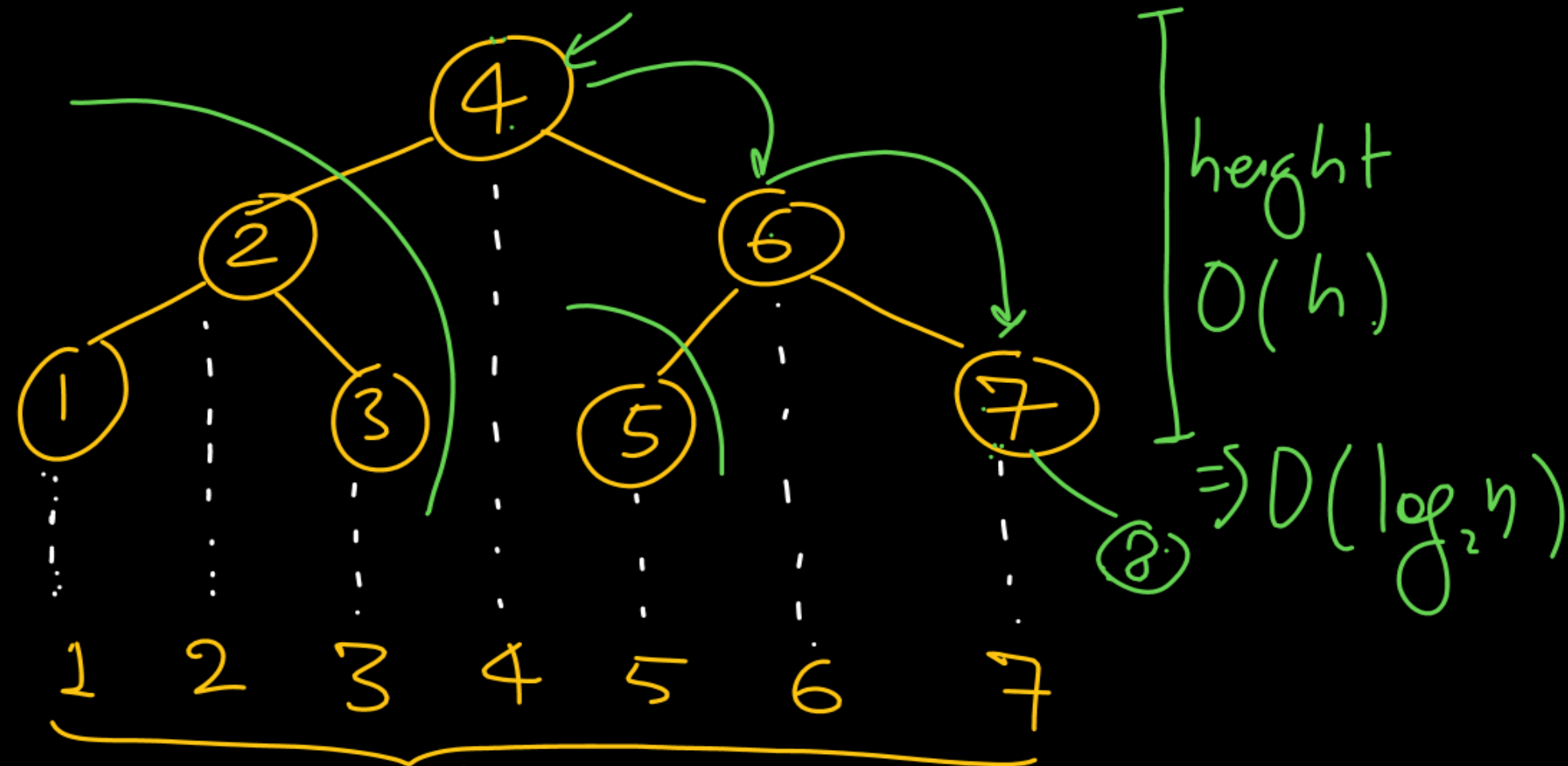
SHAIENDRA SIR

Binary Search Tree

- ✓ Binary Search Tree
- ✓ NO. of Binary Search Tree
- ✓ Operation on Binary Search Tree
- ✓ Binary Search Tree PYQ

Inorder: L Root R

Binary Search Tree : $L < \text{Root} < R$



Inorder Traversal
Always Sorted in
Ascending order: $O(n)$

Inorder
Traversal. $= \Theta(n)$

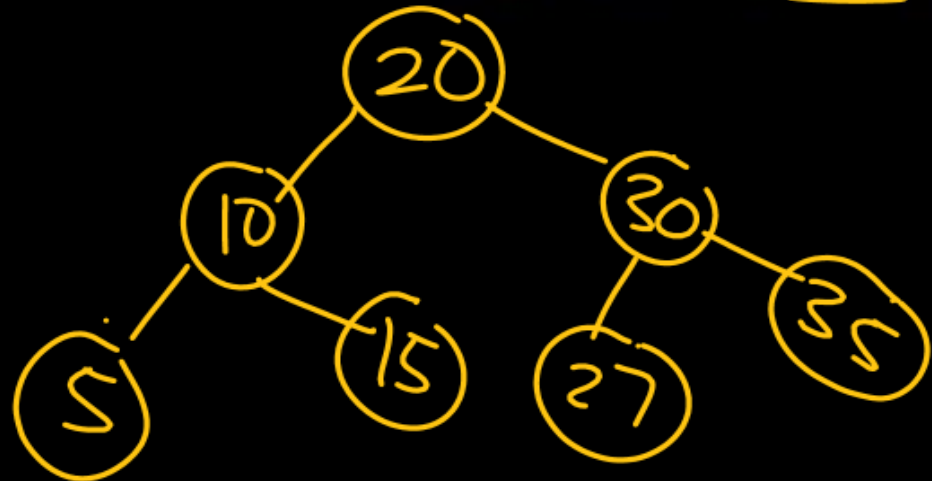
Search
Insert
Delete $\Rightarrow O(h)$
 $\Rightarrow O(\log_2 n)$

Binary Search Tree:-

- 1) BST is also a binary Tree but with restriction of all the elements in the Left Subtree should be smaller to the Parent and all the element in the Right Subtree should be greater to the Parent this rule is applicable for every node.
- 2) BST is used for searching an element
- 3) All the value in the BST must be Unique

Ex:- Construct BST, with the given keys (elements)

20, 10, 15, 30, 35, 27, 5



Inorder: Left Root Right

Preorder: Root Left Right

Postorder: Left Right Root

Inorder: 5 10 15 20 27 30 35

Preorder: 20 10 5 15 30 27 35

Postorder: 5 15 10 27 35 30 20

NOTE:-

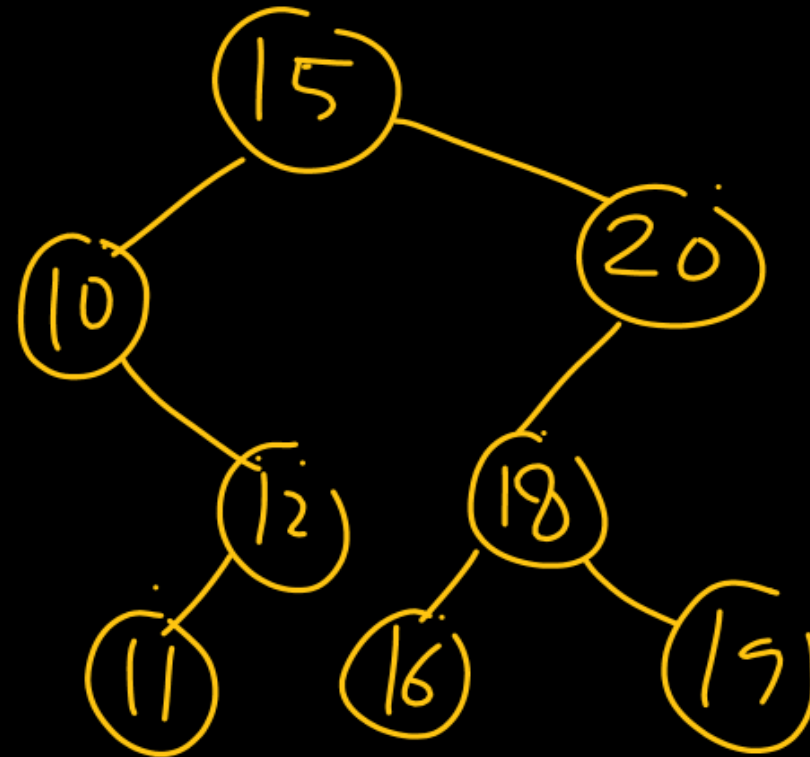
- 1) Inorder traversal of a BST is always sorted order
- 2) only Postorder or Preorder traversal of BST is given then we can construct a Unique Binary Search tree

Q) The preorder traversal of a binary search tree is ~~15~~, ~~10~~, ~~12~~, ~~11~~, ~~20~~, ~~18~~, ~~16~~, ~~19~~. Which one of the following is the post order traversal of the tree?

- ☐ A 10,11,12,15,16,18,19,20
- ☒ B 11,12,10,16,19,18,20,15
- ☐ C 20,19,18,16,15,12,11,10
- ☐ D 19,16,18,20,11,12,10,15

Pre: 15 10 12 11 20 18 16 19
In: 10 11 12 15 16 18 19 20

Post: 11 12 10 16 19 18 20 15



Number of BST:- No. of BST = No. of Unlabelled BT = Catalan No.

Number of Distinct BST Possible with n -Distinct Keys = $\frac{{}^{2n}C_n}{n+1}$

Ex:- $n = 3$, No. of Distinct BST Possible?

$$\frac{{}^{2n}C_n}{n+1} \Rightarrow \frac{{}^6C_3}{4} = \frac{6!}{4 \times 3! \times 3!} \Rightarrow \frac{\cancel{6} \times \cancel{5} \times \cancel{4} \times \cancel{3}!}{4 \times \cancel{3}! \times \cancel{3}!} = 5$$

Q) How many distinct binary search trees can be created out of 4 distinct keys?

A	5
B	14
C	24
D	35

$$n=4$$

$$\frac{2^n}{n+1} \Rightarrow \frac{2^4}{5}$$

$$b_3 = \sum_{k=0}^2 b_k \cdot b_{2-k} \Rightarrow b_0 \cdot b_2 + b_1 \cdot b_1 + b_2 \cdot b_0$$

$$\Rightarrow 1 \times 2 + 1 \times 1 + 2 \times 1 = 5$$

1. Catalan Number Representation:

$$n=3 \Rightarrow \frac{b_3}{3+1} = 5$$

$$b_n = \frac{2nC_n}{n+1}$$

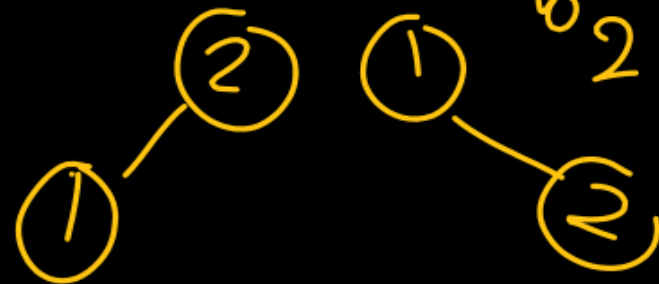
This is a variation of the Catalan number formula used to count BSTs.

2. Recursive Definition:

$$b_n = \begin{cases} 1, & n=0 \\ 1, & n=1 \end{cases}$$

$$b_n = \sum_{k=0}^{n-1} b_k \cdot b_{n-1-k}$$

$$\sum_{k=0}^{n-1} b_k \cdot b_{n-1-k}, n > 1$$



$$b_0 = 1, b_1 = 1$$

$$b_2 = \sum_{k=0}^1 b_k \cdot b_{n-1-k} \Rightarrow b_0 b_1 + b_1 b_0$$

$$\Rightarrow 1 \times 1 + 1 \times 1 = 2$$

GATE 2003

Let $T(n)$ be the number of different binary search trees on n distinct elements.

Then $T(n) = \sum_{k=1}^n T(k-1)T(x)$, where x is

A. $n - k + 1$

☒ B. $n - k$

C. $n - k - 1$

D. $n - k - 2$

$$b_n = \sum_{k=0}^{n-1} b_k \cdot b_{n-1-k}$$

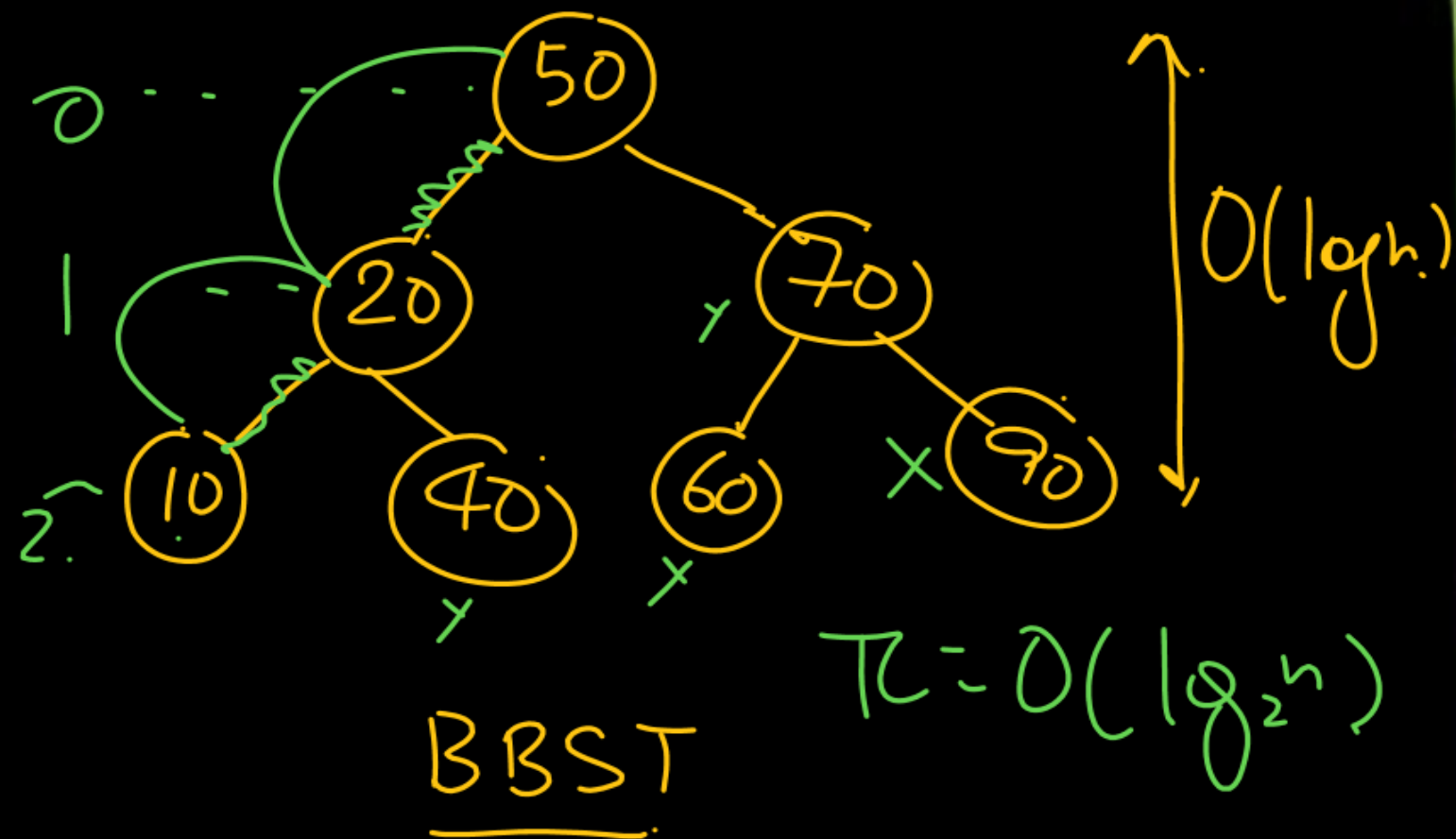
$$k-1 + 1 + x = n.$$

$$x = n - k$$

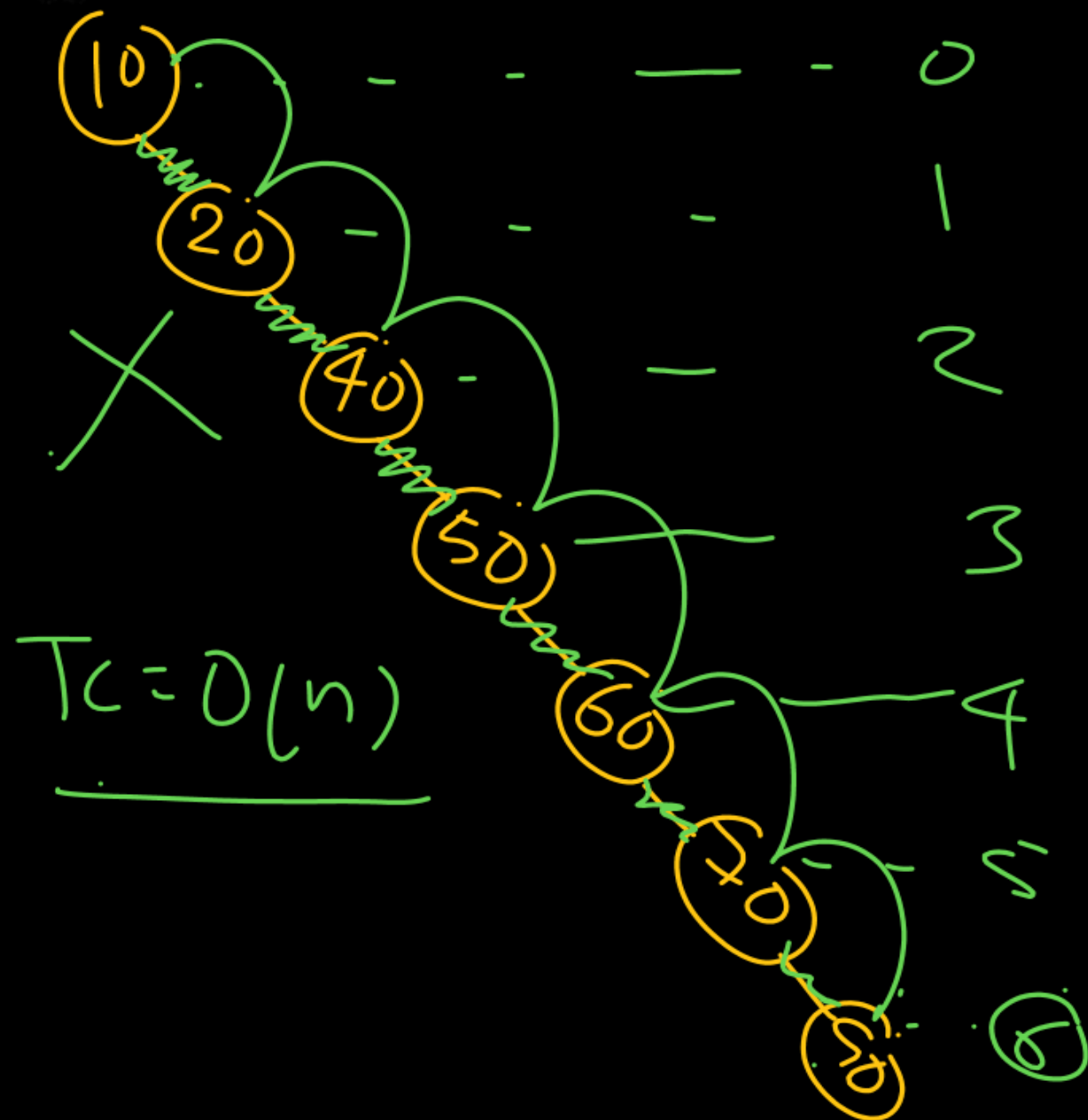
$$k + 1 + \textcircled{n-1} - k = n$$

Binary Search tree Construction:-

Keys: ~~50~~ ~~70~~ ~~60~~ ~~20~~ ~~90~~ ~~10~~ ~~40~~

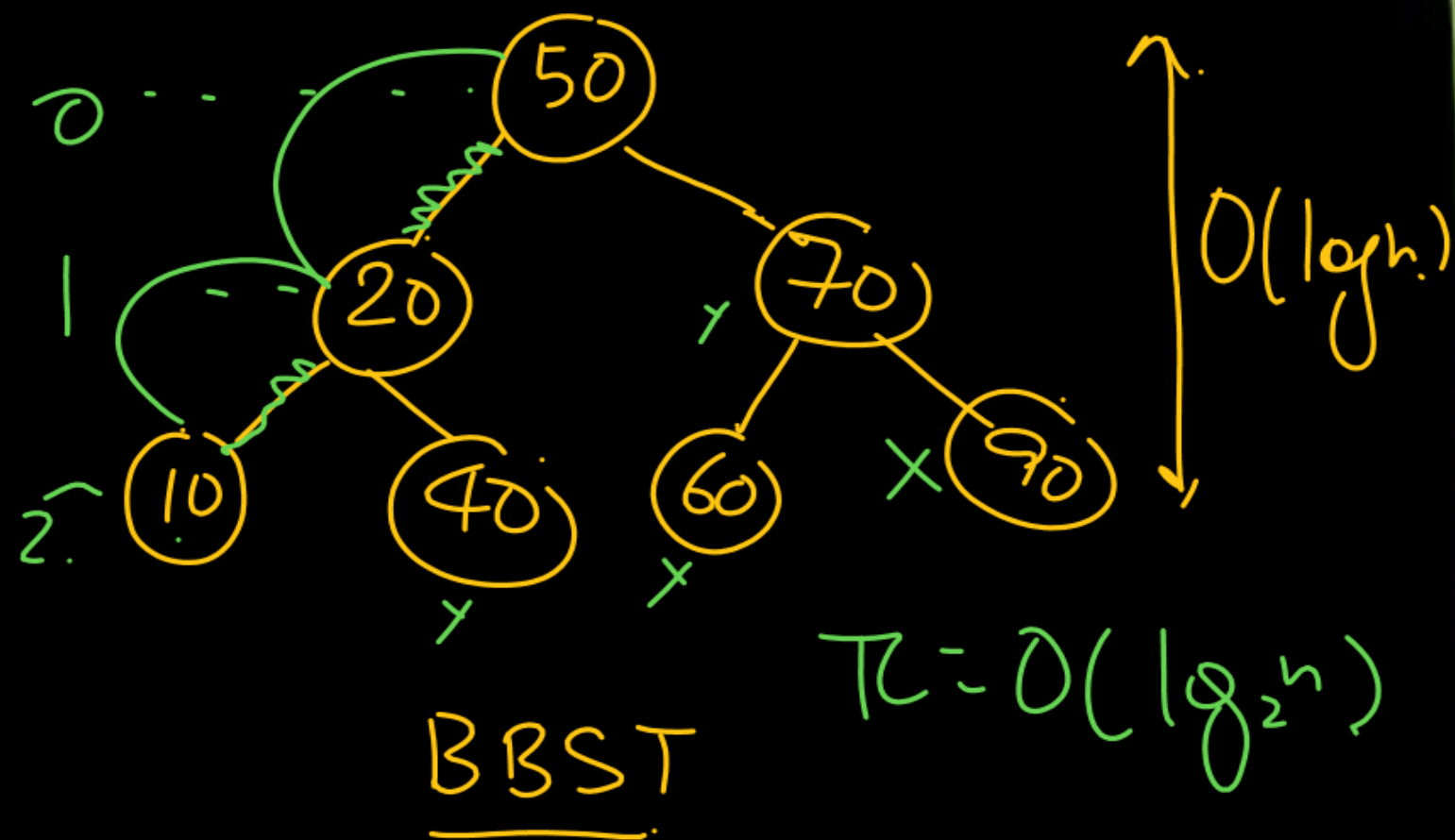


Keys: ~~10~~ ~~20~~ ~~40~~ ~~50~~ ~~60~~ ~~70~~ 90

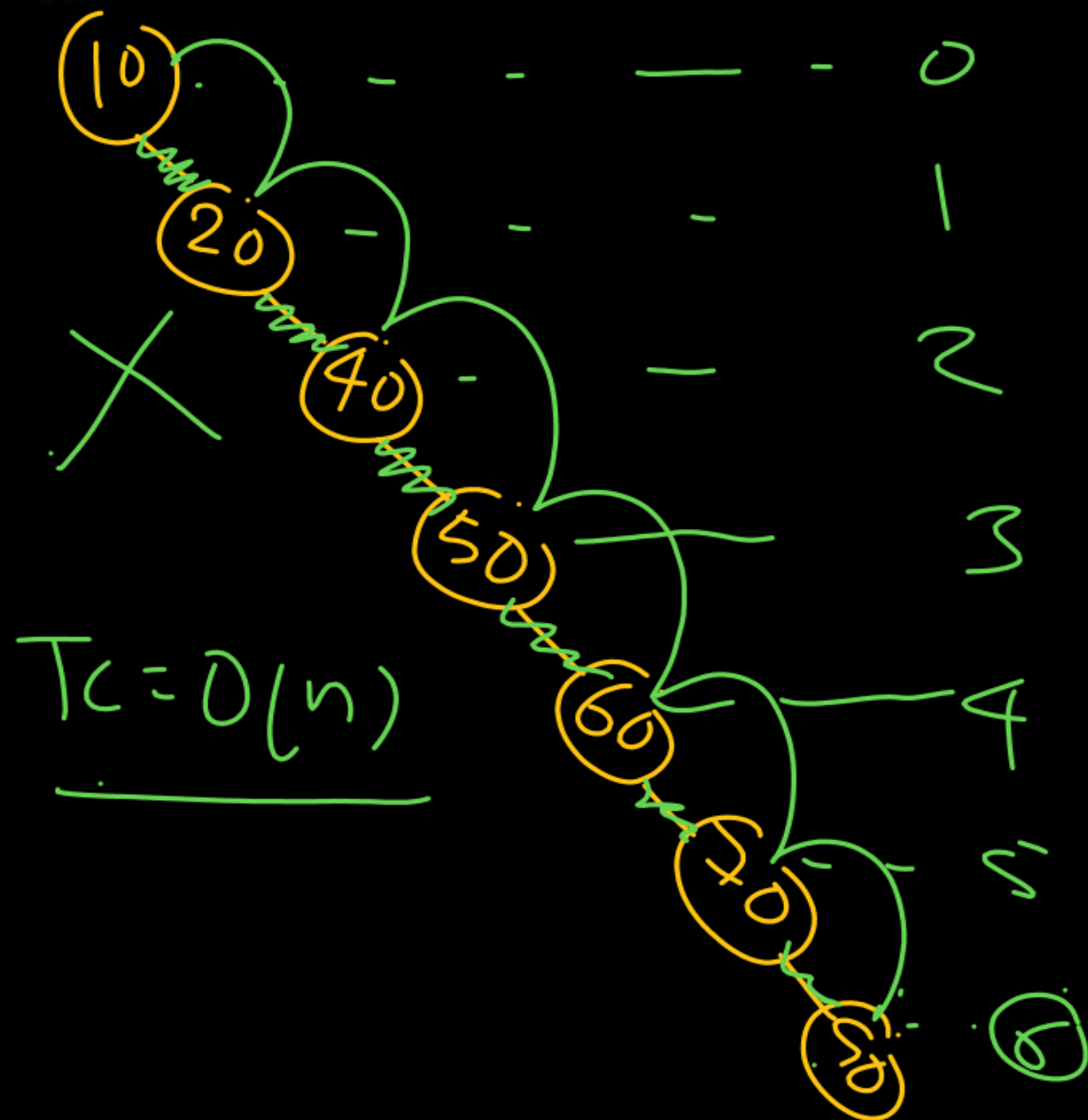


Binary Search tree Construction:-

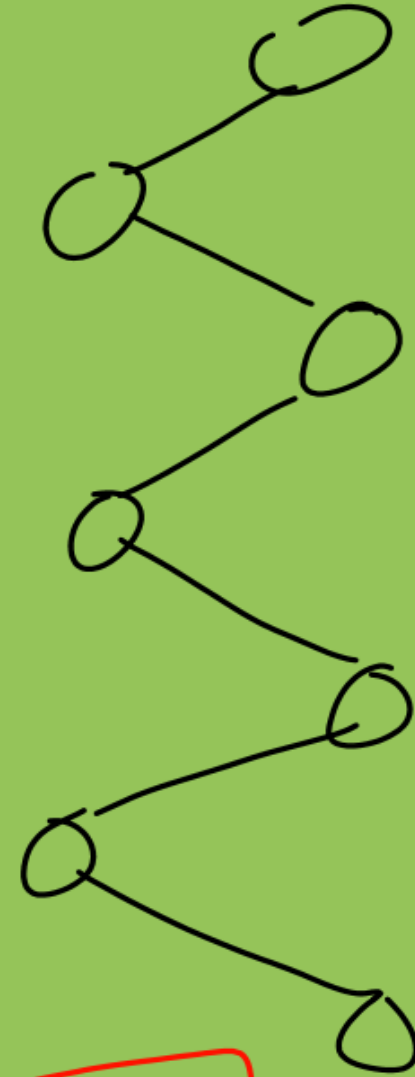
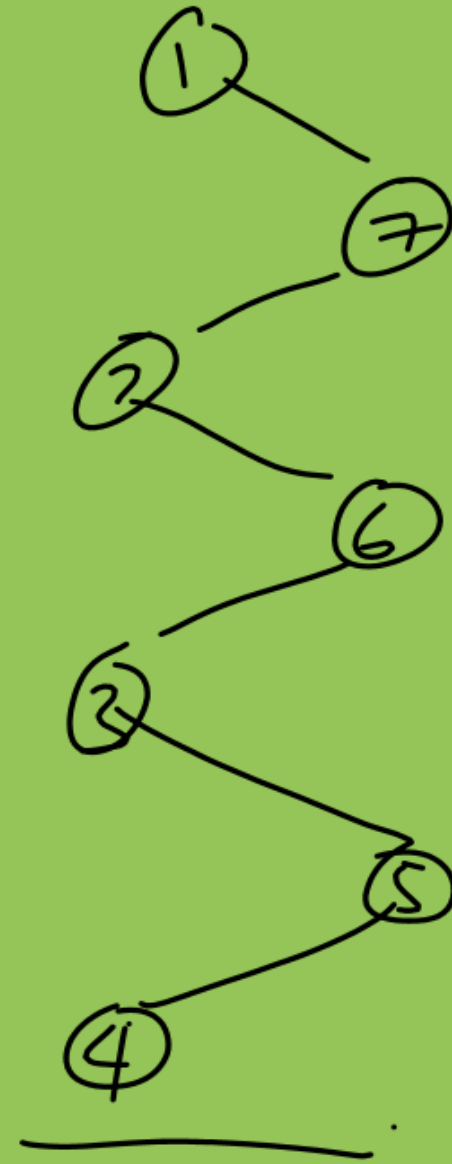
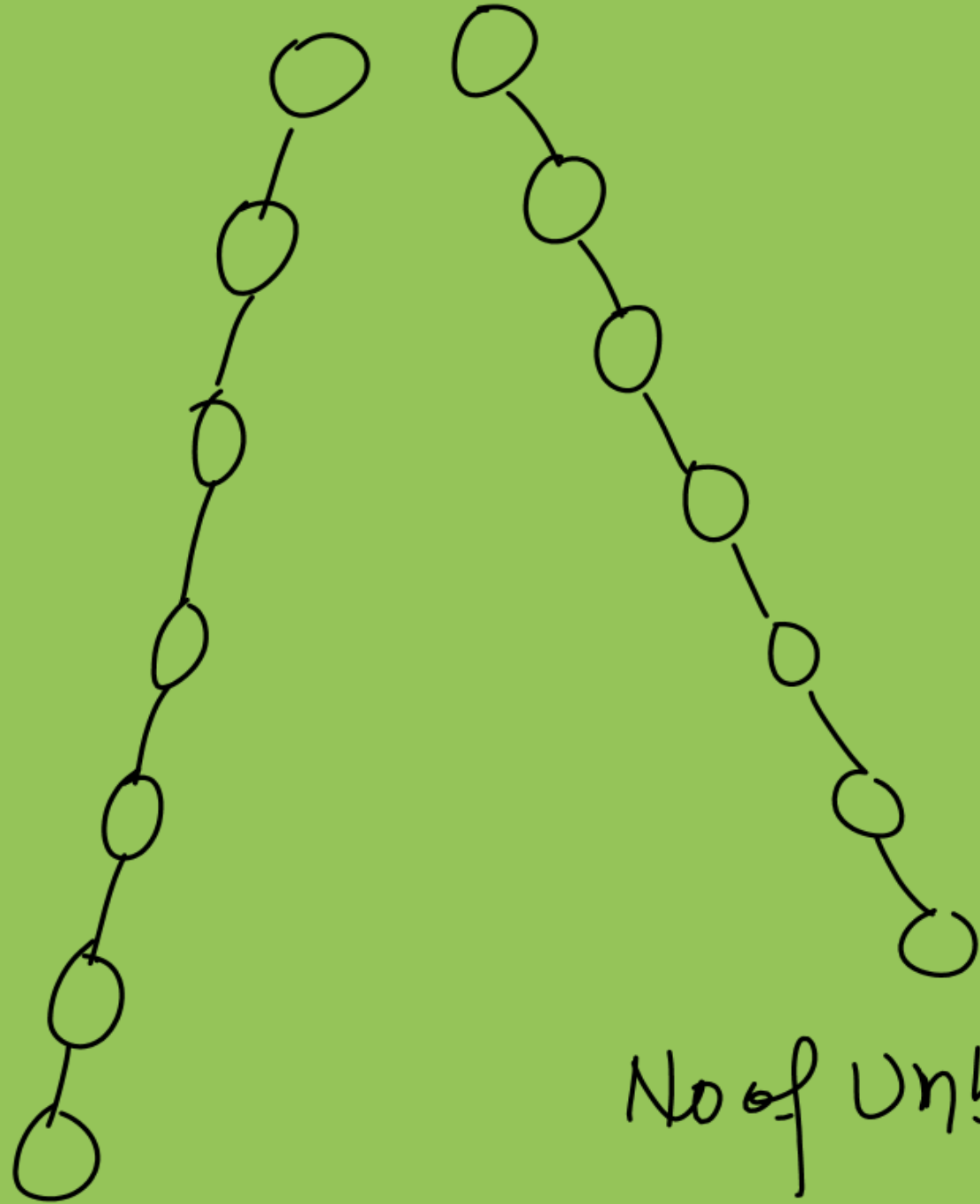
Keys: ~~50~~ ~~70~~ ~~60~~ ~~20~~ ~~90~~ ~~10~~ ~~40~~



Keys: ~~10~~ ~~20~~ ~~40~~ ~~50~~ ~~60~~ ~~70~~ 90



$n = 1, 2, 3, 4, 5, 6, 7$



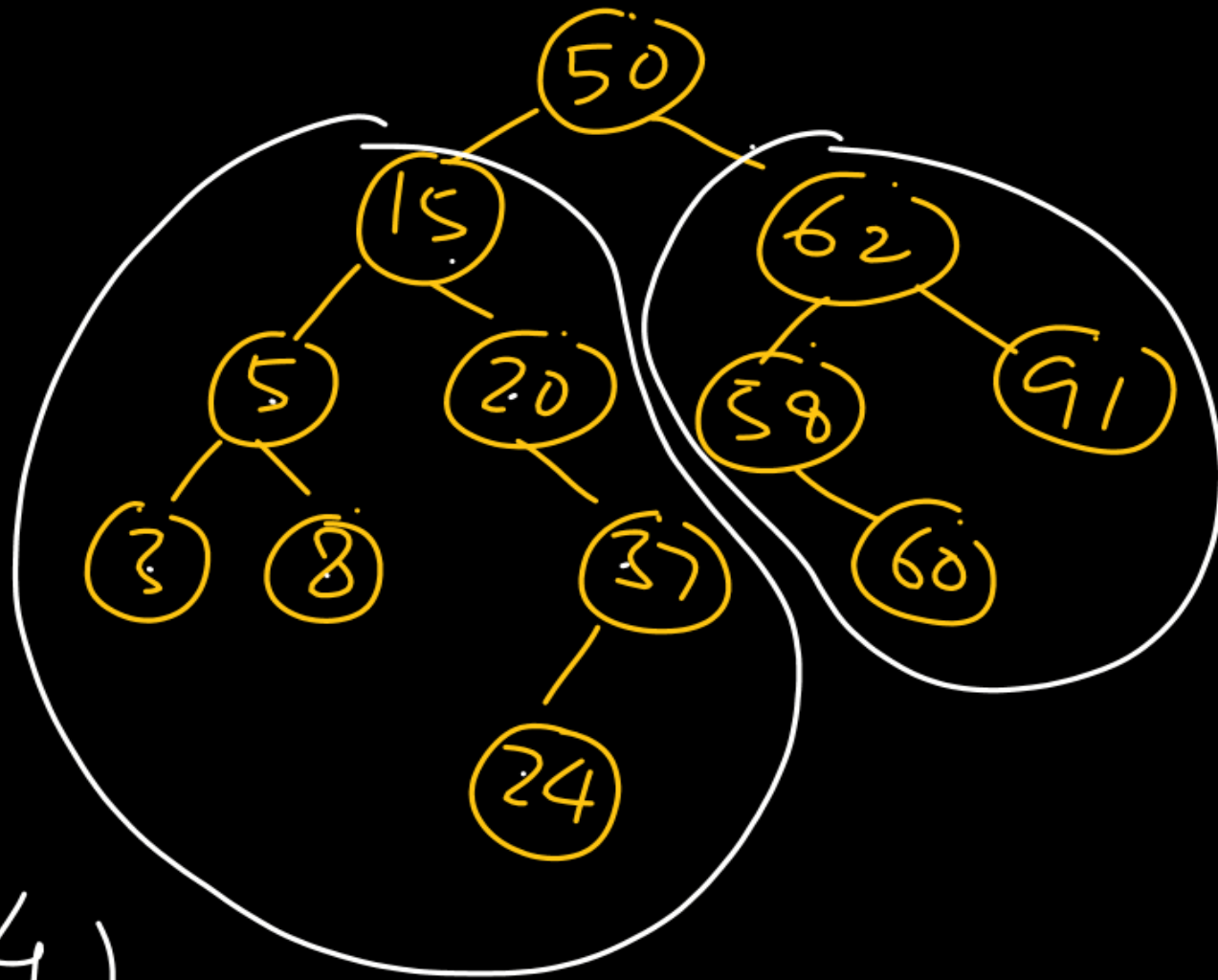
.....

$\Rightarrow$ Search
 $\Rightarrow \underline{O(n)}$

No of Unbalanced BST = 2^{n-1}

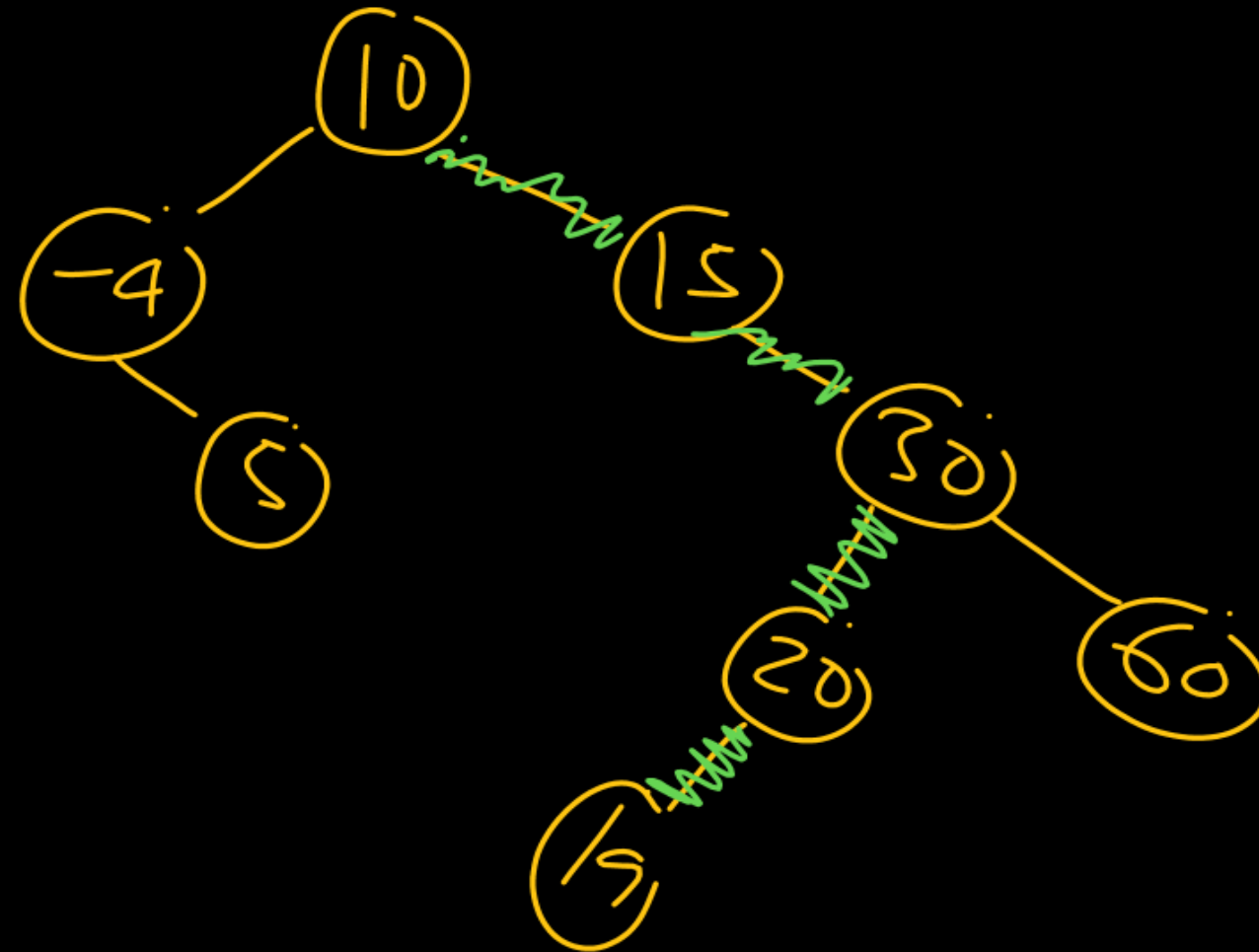
Question:- A binary search tree is generated by inserting in order of the following integers $\sqrt{50, 15, 62, 5, 20, 58, 91, 3, 8, 37, 60, 24}$ The no. of nodes in the Left Subtree and right Subtree of the root respectively is

- a) (4, 7)
- ☒ b) (7, 4)
- c) (8, 3)
- d) (3, 8)



(7, 4)

Q.35 Suppose the values ~~10~~, ~~-4~~, ~~15~~, ~~30~~, ~~20~~, ~~5~~, ~~60~~, ~~19~~ are inserted in that order into an initially empty binary search tree. Let T be the resulting binary search tree. The number of edges in the path from the node containing 19 to the root node of T is 4. (Answer in integer)



Operations on Binary Search Tree:-

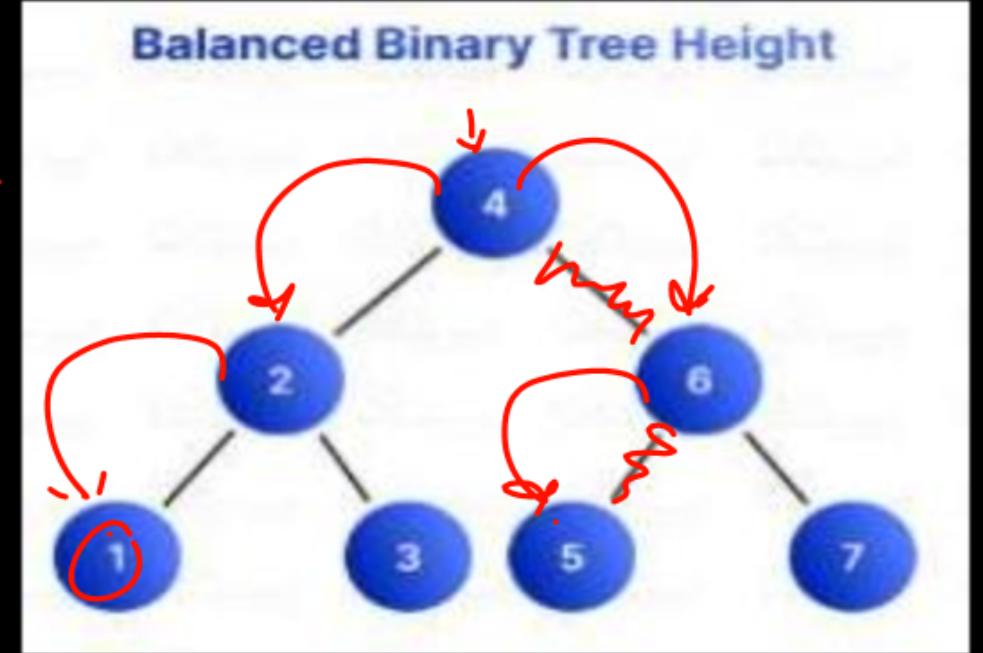
- ✓ - Search
- Insert
- Delete

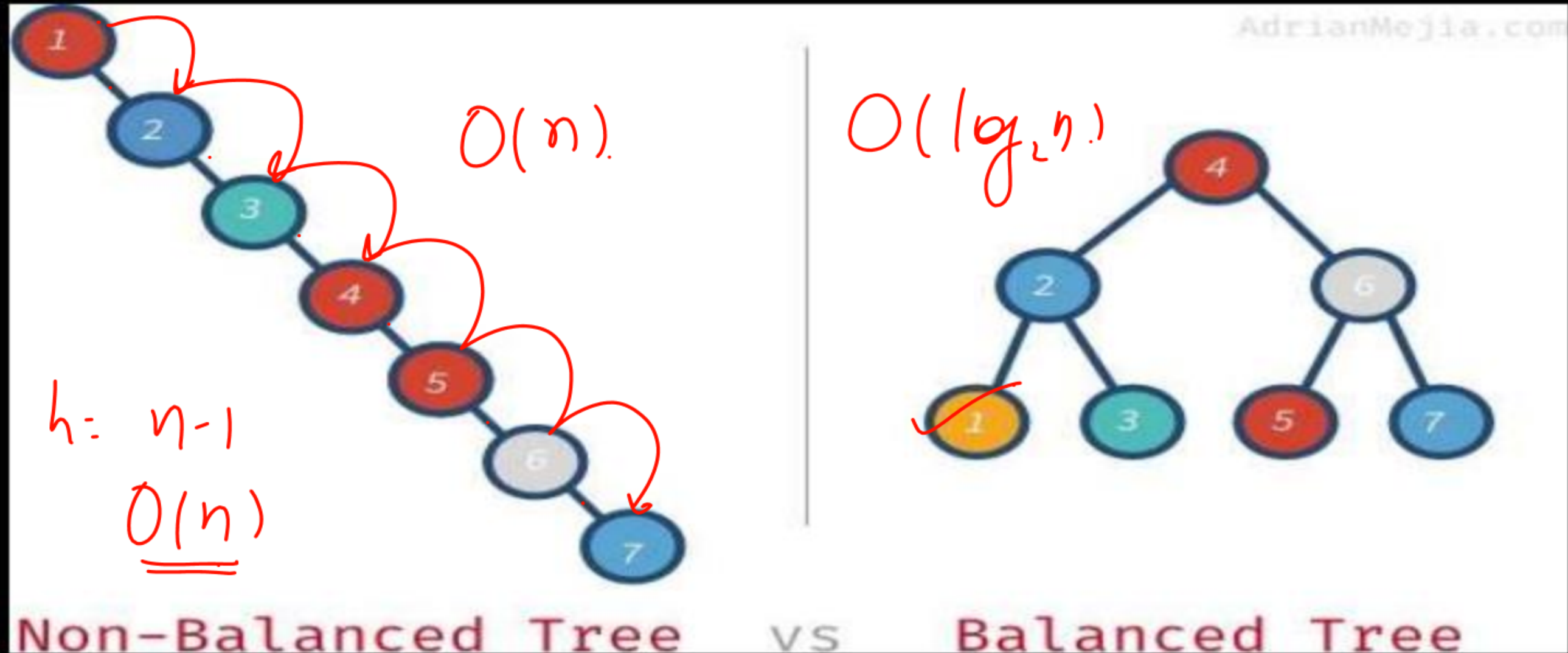
Search an element in BST:-

How the Search Works

- ✓ 1. Start at the root node.
- ✓ 2. If the key matches the root, return **True**.
- ✓ 3. If the key is **smaller**, search in the **left** subtree.
- ✓ 4. If the key is **greater**, search in the **right** subtree.
- ✓ 5. If a leaf node is reached without finding the key, return **False**.

$O(h)$





NOTE: The maximum length of a path from the root node to any other node is $(n - 1)$

NOTE: Worst case search time complexity in BST = $O(n)$

Node * Search (Node * t, int x)

{ if (t == NULL)

return NULL;

if (x == t->data)

return t;

if (x < t->data)

return Search(t->left, x);

else

return Search(t->right, x);

}

BST:

Search

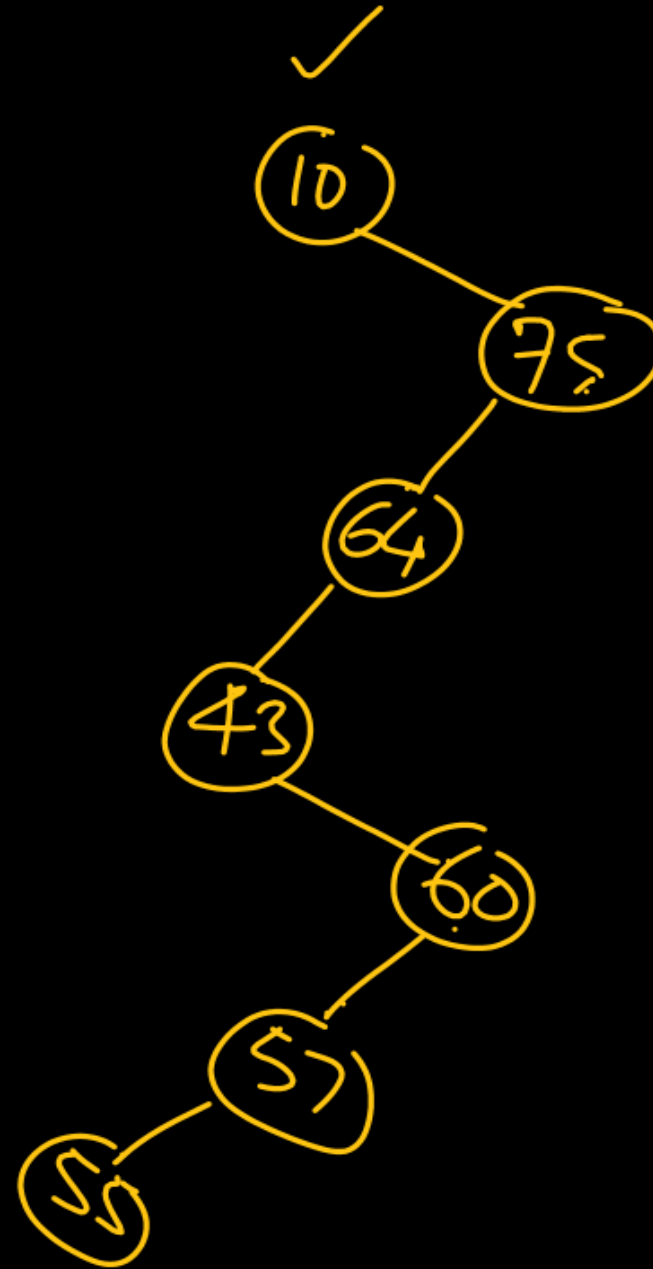
$$BC = O(1)$$

$$AC = O(\log_2 n)$$

$$WC = O(n)$$

Q) Suppose that we have numbers between 1 and 100 in a binary search tree and want to search for the number 55. Which of the following sequences CANNOT be the sequence of nodes examined?

- | | |
|---|------------------------|
| A | {10,75,64,43,60,57,55} |
| B | {90,12,68,34,62,45,55} |
| C | {9,85,47,68,43,57,55} |
| D | {79,14,72,56,16,53,55} |



Q) Suppose that we have numbers between 1 and 100 in a binary search tree and want to search for the number 55. Which of the following sequences CANNOT be the sequence of nodes examined?

- A {10, 75, 64, 43, 60, 57, 55}
- B {90, 12, 68, 34, 62, 45, 55}
- ☒ C {9, 85, 47, 68, 43, 57, 55}
- D {79, 14, 72, 56, 16, 53, 55}

10, 43

75, 64, 60, 57

9, 47, 43

85, 68, 57

} X

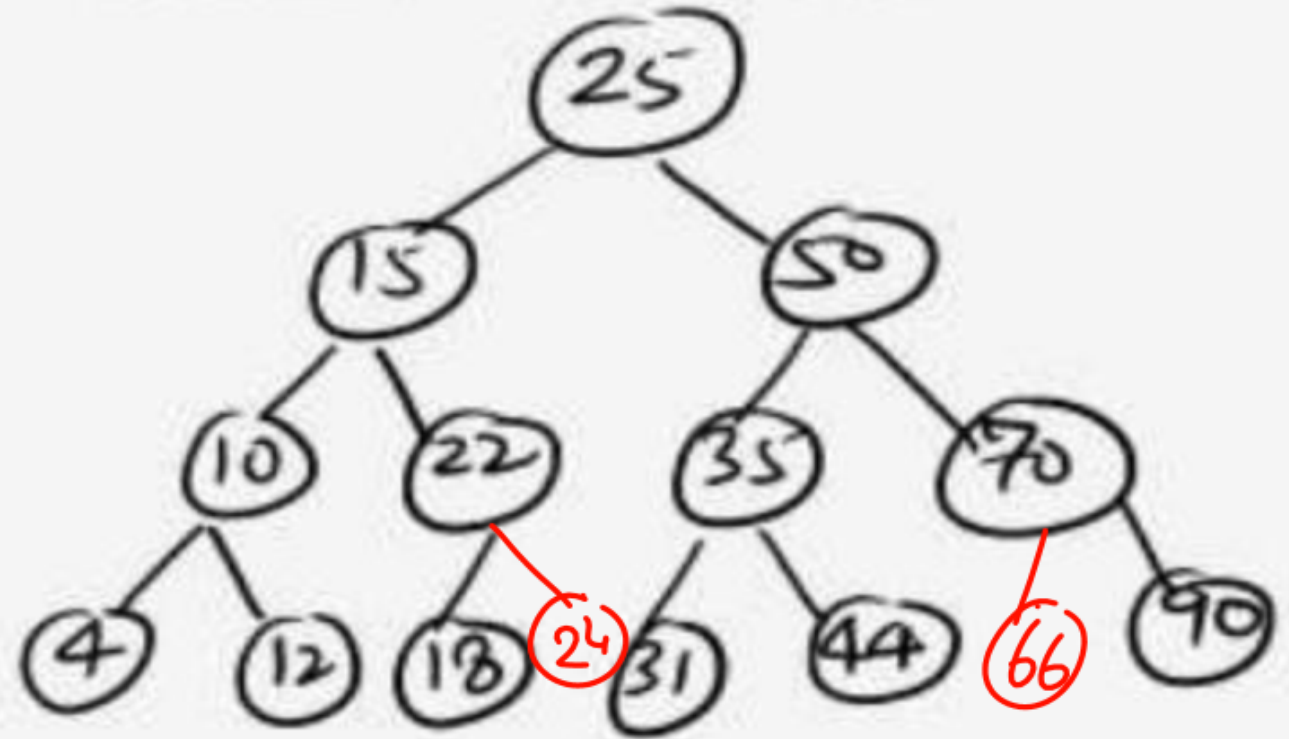
Insertion in BST

- ✓ Start at the root.
- ✓ If the new key is smaller, go to the left subtree.
- ✓ If the new key is greater, go to the right subtree.
- ✓ Insert when an empty spot is found.

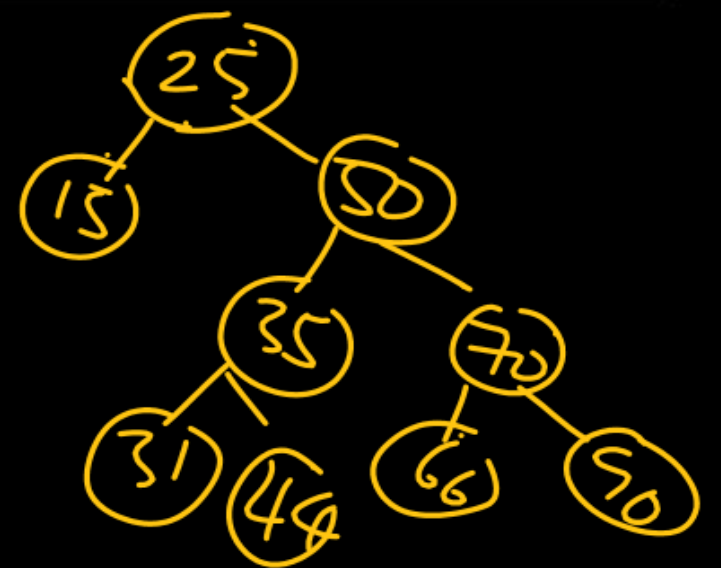
Time Complexity:

$$\begin{array}{l} \text{Search} = O(h) \\ \text{Add} = O(1) \\ \hline O(h) \end{array}$$

Insert:- 24, 66



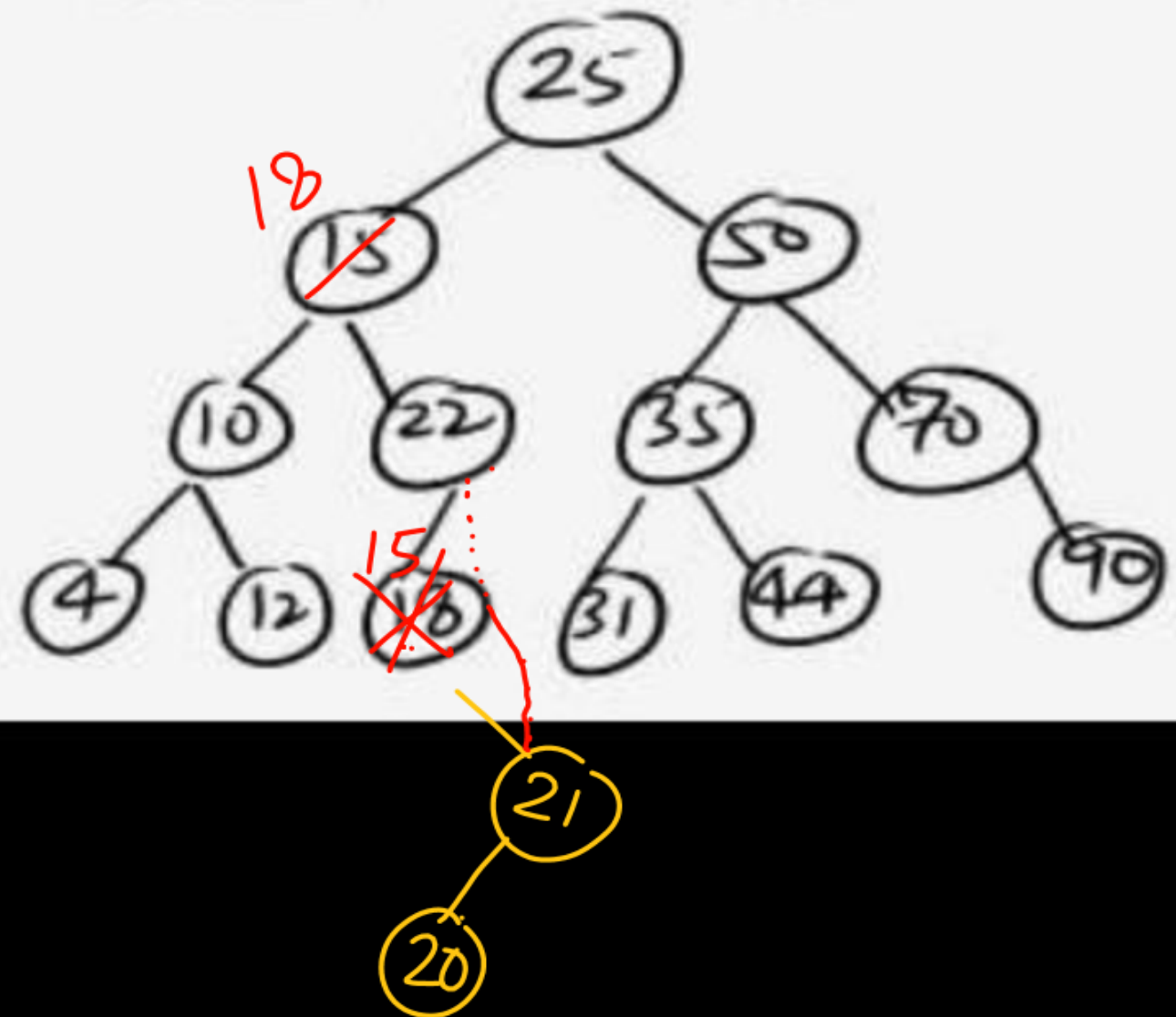
$$\begin{array}{l} \text{BC} = O(1) \\ \text{AC} = O(\log_2 n) \\ \text{WC} = O(n) \end{array}$$



Insertion in BST

- ✓ Start at the root.
- ✓ If the new key is **smaller**, go to the left subtree.
- ✓ If the new key is **greater**, go to the right subtree.
- ✓ Insert when an empty spot is found.

Insert:- 24, 66

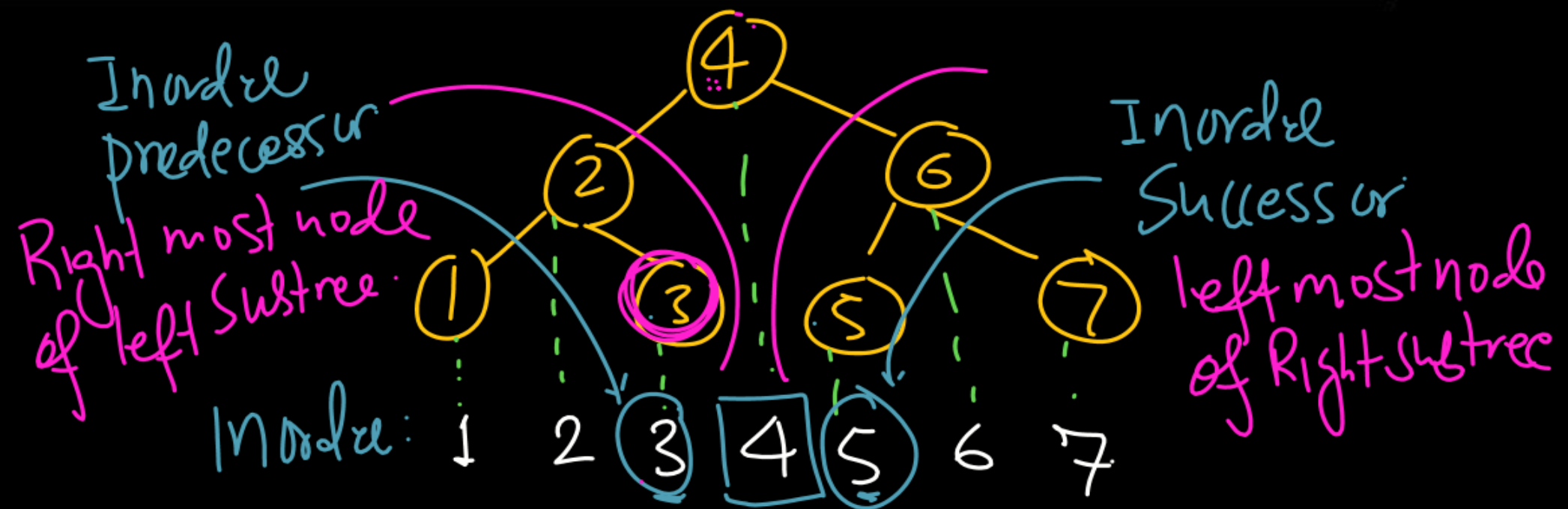


Delete 15

Deletion in BST

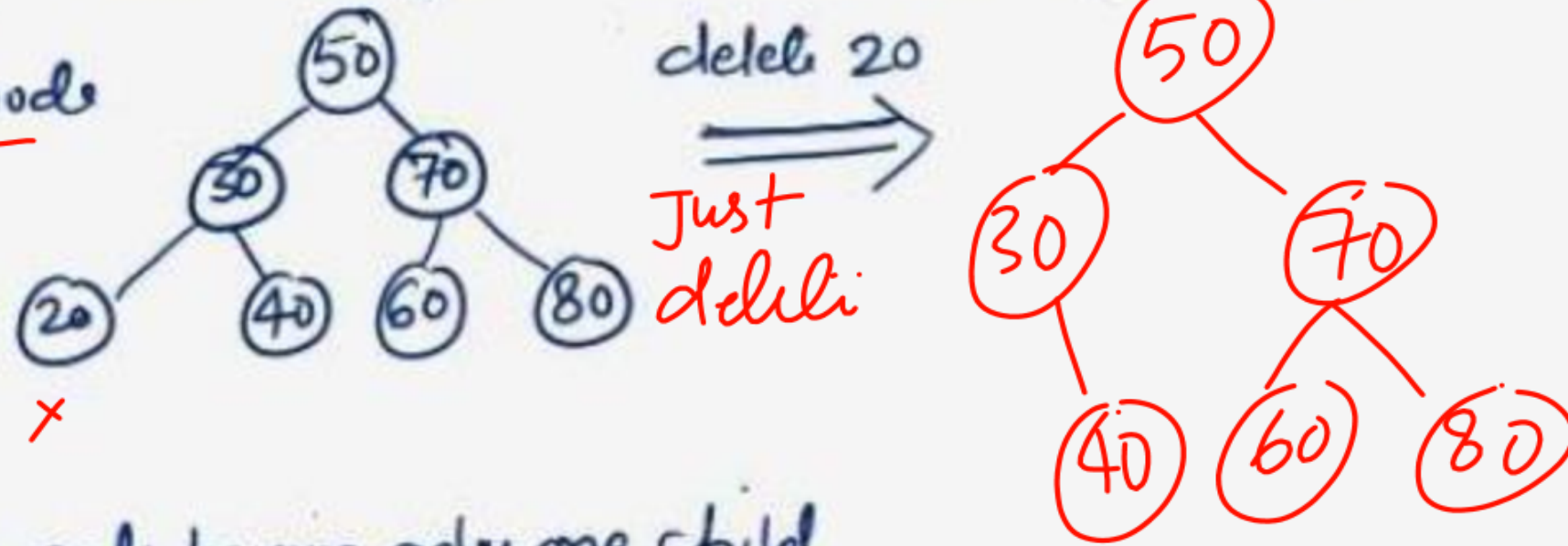
Three Cases for Deletion:

1. Node has no children (leaf node): Simply remove it.
2. Node has one child: Replace it with its child.
3. Node has two children: Find the inorder successor (smallest node in right subtree), replace node's value, and delete the successor.

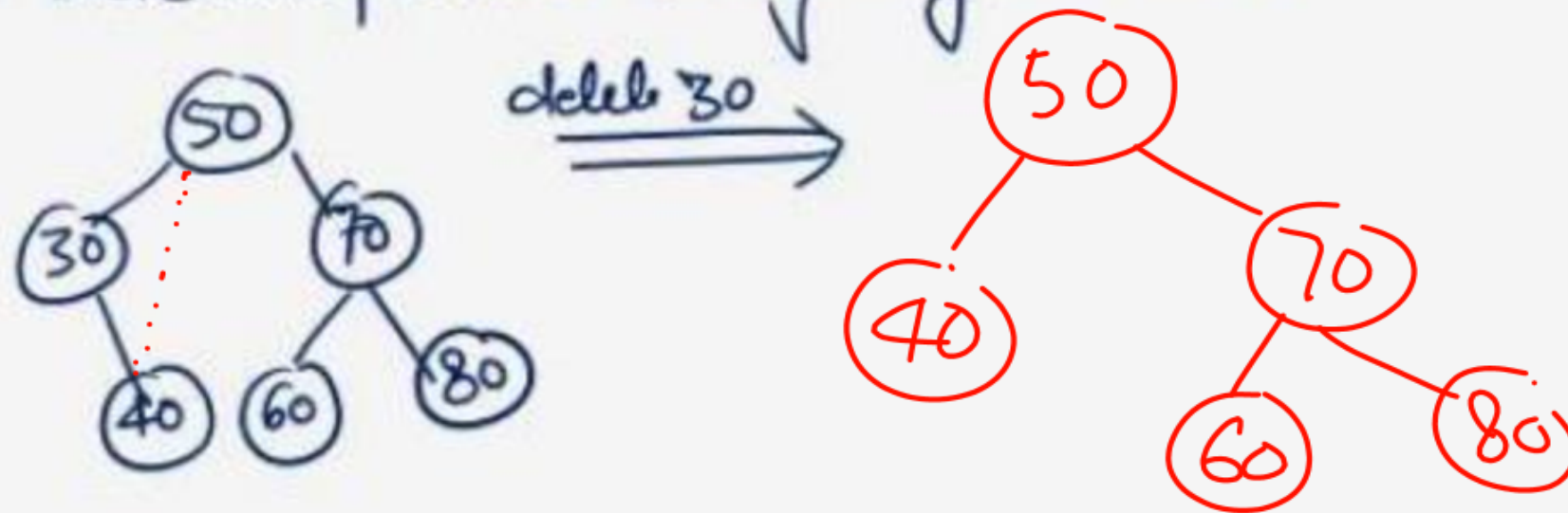


Delete:- When it comes to deleting a node from the binary search tree, following three cases are possible

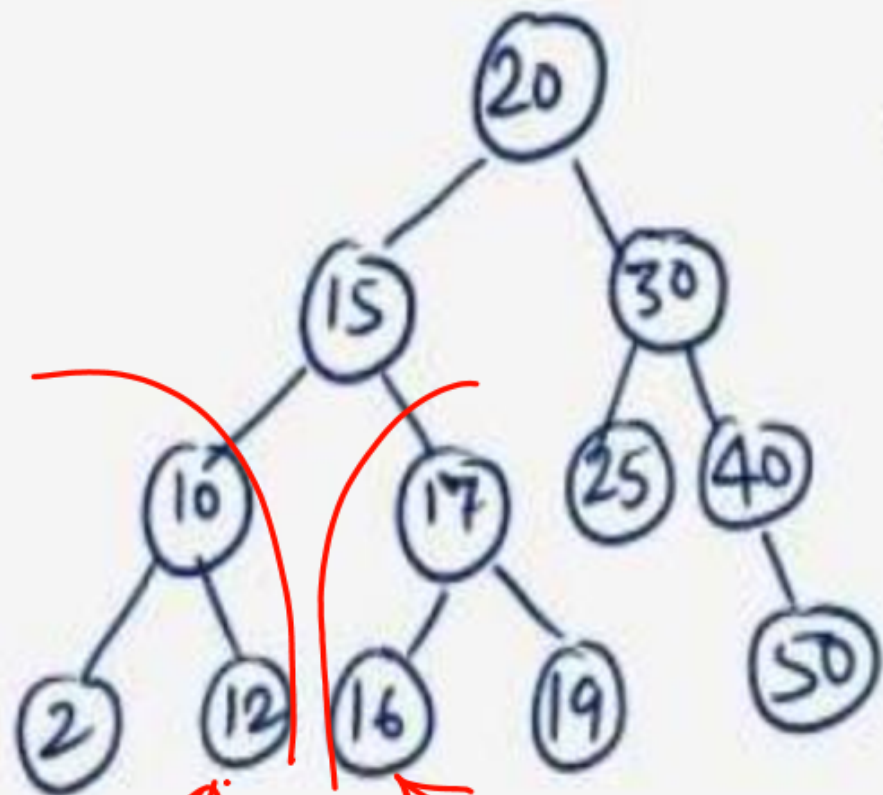
Case 1:- Delete Leaf node



Case 2: Deletion of a node having only one child



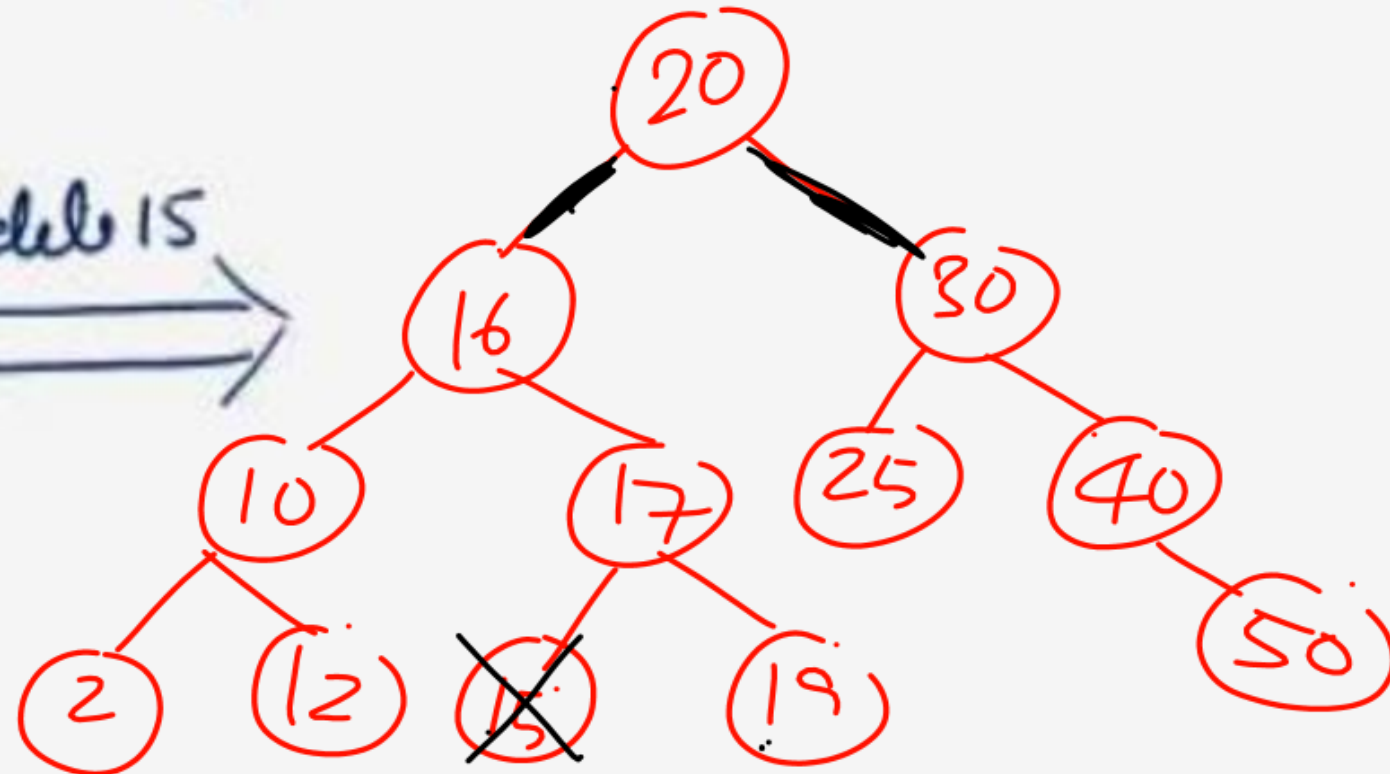
Case 3:- Deletion of a node having two children



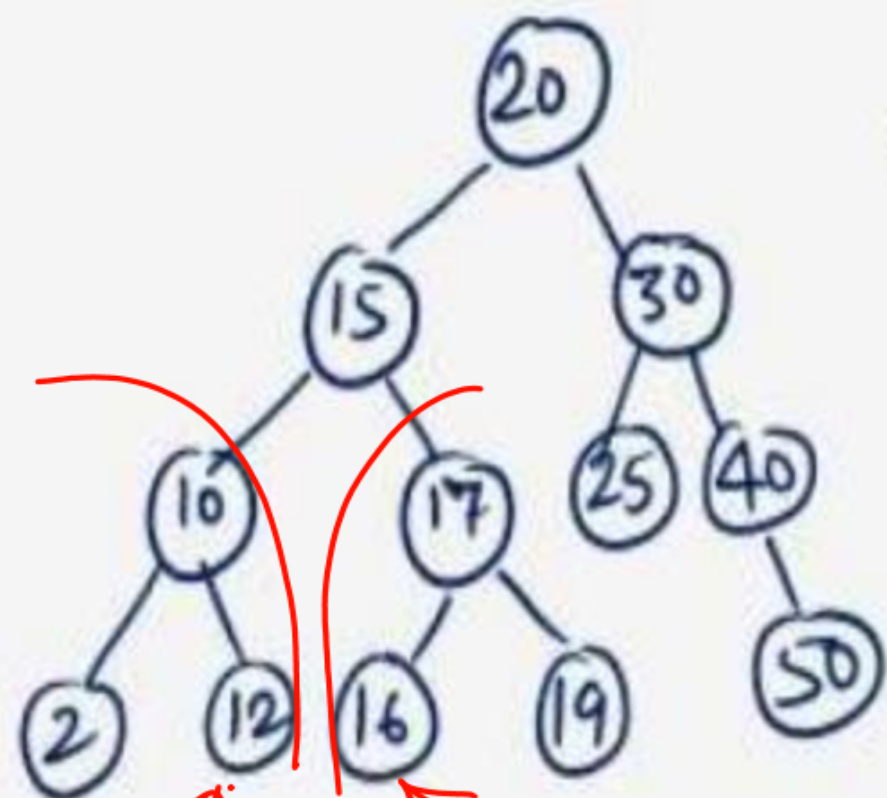
Inorder
Predecessor

Inorder
Successor

delete 15
⇒



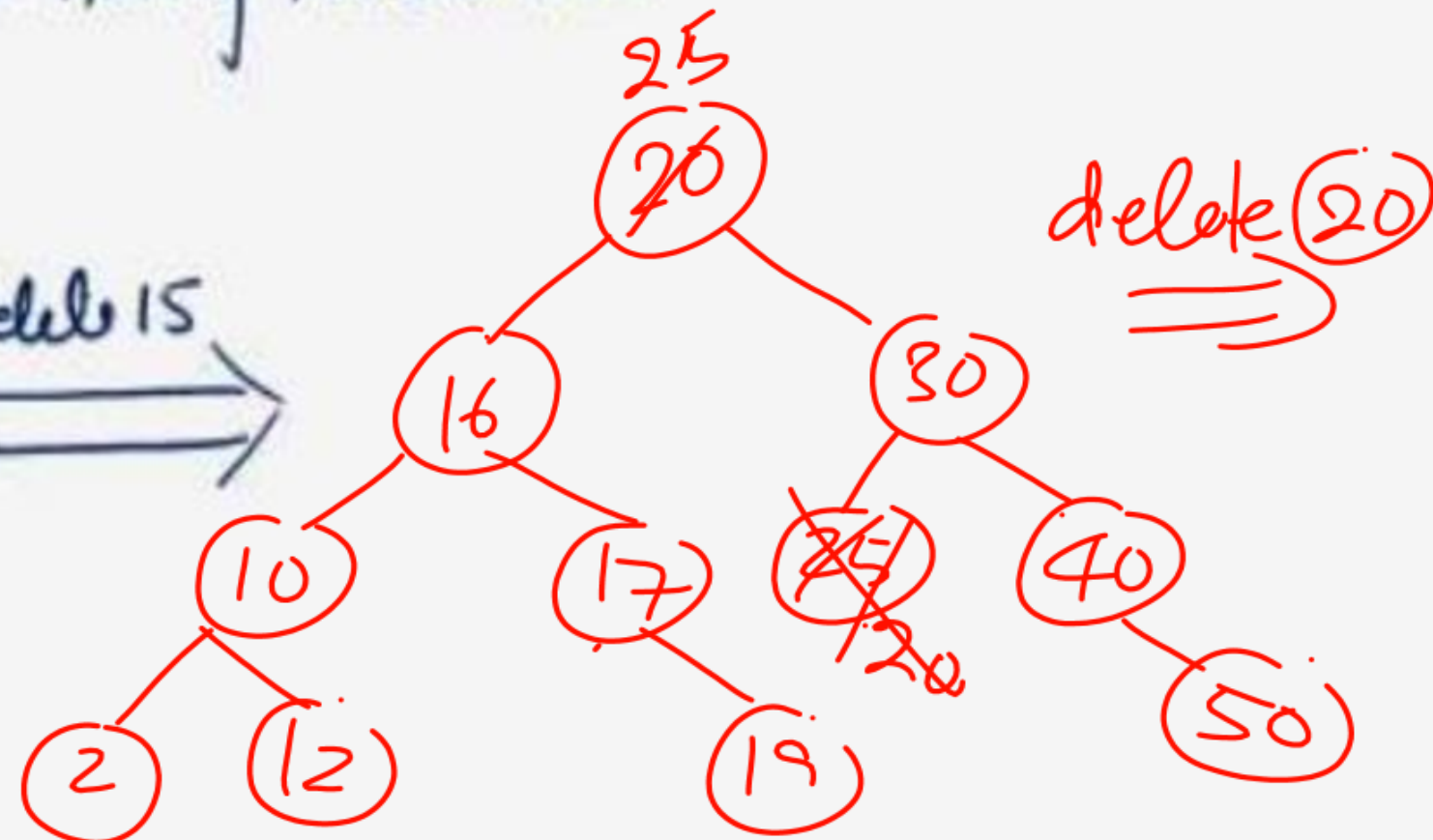
Case 3:- Deletion of a node having two children



Inorder
Predecessor

Inorder
Successor

delete 15
⇒



Deletion:

Search $\Rightarrow O(h)$

Inorder
Successor / Inorder
Predecessor $= O(h)$

Delete $= O(1)$

$O(h)$

Best $= O(1)$

Avg $= O(\log n)$

WC $= O(h)$

WC $= O(h)$

$\Downarrow$ Reduces to

$O(\log_2 n) \Rightarrow$ AVL Tree

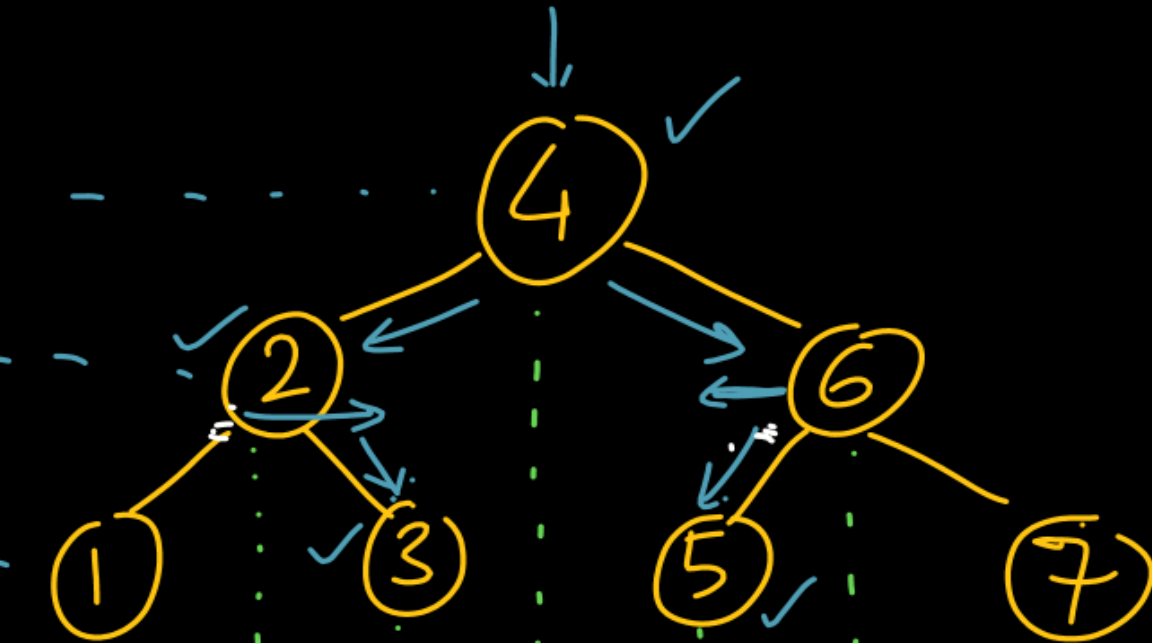
Range Search in a BST

if Range = $[3, 5]$

Total nodes Explored.
or Search = $O(h+m)$

height
of tree

2



[1 2 [3 4 5] 6 7]

No of nodes in the Range = $m = 3$
height of tree = 2.

if Range = $[1, 7]$

No of nodes in the Range = $m = 7$

Total search Time = $O(m)$

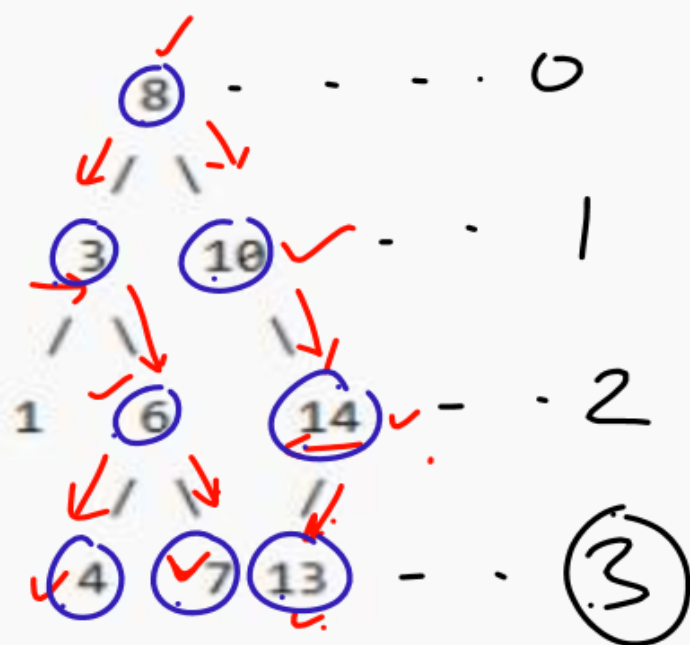
Example

Consider the following BST:

markdown

 Copy

 Edit



$$\text{TotalTime} = O(h + m)$$

$$= O(3 + 5) = \underline{\underline{8}}$$

To find all keys in the range [5, 13], the algorithm would return: [6, 7, 8, 10, 13]. $m = \underline{\underline{5}}$

Range Search in a BST

🧠 Time and Space Complexity

- **Time Complexity:** In a balanced BST, the time complexity is $O(h + k)$, where h is the height of the tree and k is the number of nodes within the range. In the worst-case scenario (e.g., a skewed tree), the time complexity can degrade to $O(n)$, where n is the total number of nodes.
- **Space Complexity:** The space complexity is $O(h)$ due to the recursion stack during traversal. HeyCoach Blog

$$O(h+k) \begin{cases} \text{Balanced} = \underline{\underline{O(\log_2 n + k)}} \\ \text{Skewed} = O(n+k) \Rightarrow \underline{\underline{O(n)}} \end{cases}$$

Suppose we have a balanced binary search tree T holding n numbers. We are given two numbers L and H and wish to sum up all the numbers in T that lie between L and H . Suppose there are m such numbers in T . If the tightest upper bound on the time to compute the sum is $O(n^a \log^b n + m^c \log^d n)$, the value of $a + 10b + 100c + 1000d$ is _____.

Range: $[L, H]$ $k = m$.

$$TC = O(\log_2 n + m)$$

$$TC = O(n^a \log_2^b n + m^c \log_2^d n)$$

$$a=0, b=1, c=1, d=0$$

$$a + 10b + 100c + 1000d \quad \underline{110} ?$$

$$0 + 10 \times 1 + 100 \times 1 + 1000 \times 0$$

$$\Rightarrow 10 + 100 = \underline{110}$$

Ans

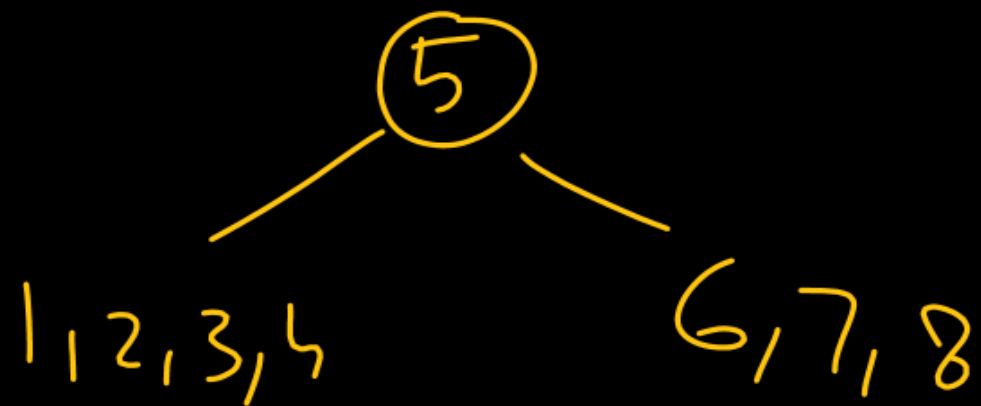
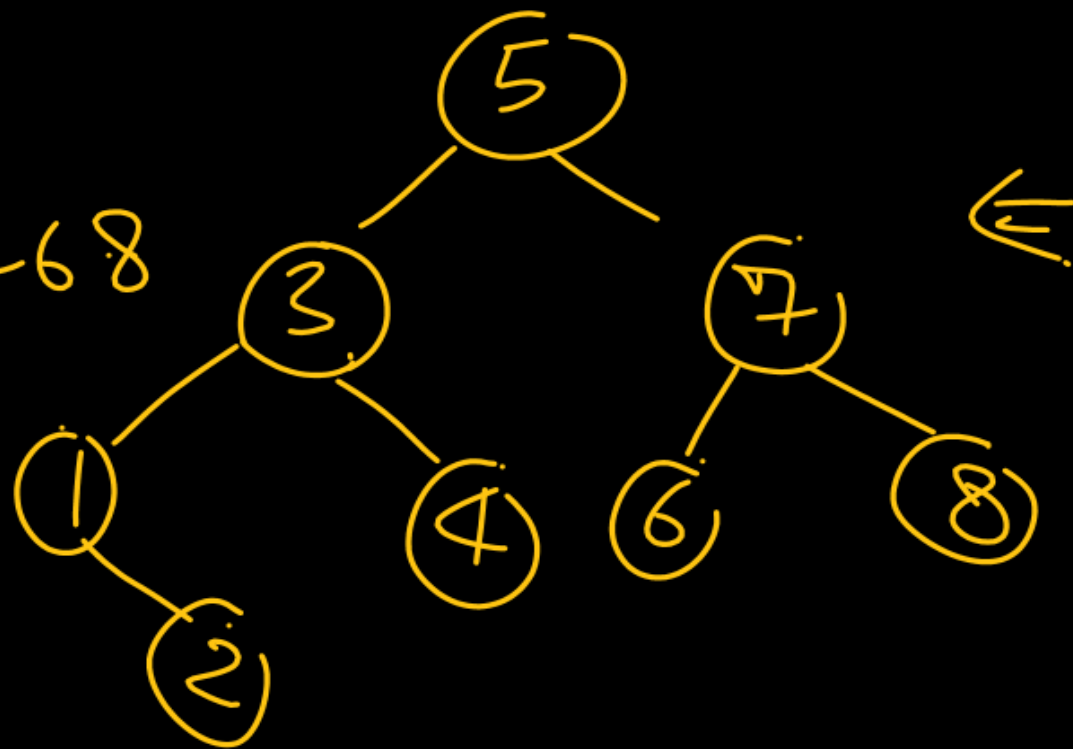
A binary search tree contains the value 1, 2, 3, 4, 5, 6, 7, 8. The tree is traversed in pre-order and the values are printed out. Which of the following sequences is a valid output?

- ☒ A. 5 3 1 2 4 7 8 6
- ☒ B. 5 3 1 2 6 4 8 7
- ☒ C. 5 3 2 4 1 6 7 8
- ☒ D. 5 3 1 2 4 7 6 8

Pre: Root Left Right

Preorder

5 3 1 2 4 7 6 8





Shailendra Singh GATE CSE/DA

THANKYOU



GeeksforGeeks GATE